

Assignment 8: Time Series Analysis

Student Name

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on generalized linear models.

Directions

1. Rename this file `<FirstLast>_A08_TimeSeries.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up

1. Set up your session:
 - Check your working directory
 - Load the tidyverse, lubridate, zoo, and trend packages
 - Set your ggplot theme

```
# Check working directory
getwd()
```

```
## [1] "/Users/kaitlynkukula/EDE_Fall2025"
```

```
# Load packages
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr    1.5.1
## v ggplot2    4.0.0      v tibble     3.3.0
## v lubridate  1.9.4      v tidyr      1.3.1
## v purrr      1.1.0
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```

library(lubridate)
library(zoo)

##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##      as.Date, as.Date.numeric

library(trend)
library(ggplot2)

# Set ggplot theme
kak_theme <- theme_classic() +
  theme(plot.title = element_text(size = 11, hjust = 0.5, face = "bold"), #Adjust title
        axis.text.x = element_text(size = 9), # Adjust x-axis values
        axis.text.y = element_text(size = 9), # Adjust y-axis values
        axis.title.x = element_text(size = 10.5, face = "bold"), # Adjust x-axis title
        axis.title.y = element_text(size = 10.5, face = "bold"), # Adjust y-axis title
        legend.position = "right", # Define legend position
        legend.text = element_text(size = 9), # Define legend entry sizes
        legend.title = element_text(size = 9)) # Define legend name sizes
set_theme(kak_theme)

```

2. Import the ten datasets from the Ozone_TimeSeries folder in the Raw data folder. These contain ozone concentrations at Garinger High School in North Carolina from 2010-2019 (the EPA air database only allows downloads for one year at a time). Import these either individually or in bulk and then combine them into a single dataframe named `GaringerOzone` of 3589 observation and 20 variables.

```

#1: Loading data

# Import datasets as data frames
garinger_ozone10 <- data.frame(read.csv(file = "Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2010_raw.
garinger_ozone11 <- data.frame(read.csv(file = "Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2011_raw.
garinger_ozone12 <- data.frame(read.csv(file = "Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2012_raw.
garinger_ozone13 <- data.frame(read.csv(file = "Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2013_raw.
garinger_ozone14 <- data.frame(read.csv(file = "Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2014_raw.
garinger_ozone15 <- data.frame(read.csv(file = "Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2015_raw.
garinger_ozone16 <- data.frame(read.csv(file = "Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2016_raw.
garinger_ozone17 <- data.frame(read.csv(file = "Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2017_raw.
garinger_ozone18 <- data.frame(read.csv(file = "Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2018_raw.
garinger_ozone19 <- data.frame(read.csv(file = "Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2019_raw.

# Combing into single dataframe
garinger_ozone <- rbind(garinger_ozone10, garinger_ozone11, garinger_ozone12,
                        garinger_ozone13, garinger_ozone14, garinger_ozone15,
                        garinger_ozone16, garinger_ozone17, garinger_ozone18,
                        garinger_ozone19)

```

Wrangle

3. Set your date column as a date class.

4. Wrangle your dataset so that it only contains the columns Date, Daily.Max.8.hour.Ozone.Concentration, and DAILY_AQI_VALUE.
5. Notice there are a few days in each year that are missing ozone concentrations. We want to generate a daily dataset, so we will need to fill in any missing days with NA. Create a new data frame that contains a sequence of dates from 2010-01-01 to 2019-12-31 (hint: `as.data.frame(seq())`). Call this new data frame Days. Rename the column name in Days to “Date”.
6. Use a `left_join` to combine the data frames. Specify the correct order of data frames within this function so that the final dimensions are 3652 rows and 3 columns. Call your combined data frame GaringerOzone.

```
#3: Set date column as a date
garinger_ozone$Date <- mdy(garinger_ozone$Date)

#4: Select Date, Ozone Concentration, and AQI
garinger_ozone_select <- garinger_ozone %>%
  select(Date, Daily.Max.8.hour.Ozone.Concentration, DAILY_AQI_VALUE)

#5: New dataframe that covers 2010-01-01 to 2019-12-31
days <- as.data.frame(seq(as.Date('2010-01-01'), as.Date('2019-12-31'), by = 'day'))
colnames(days) <- "Date"

#6: Join to combine the dates and ozone concentration dataframes
garinger_ozone <- left_join(days, garinger_ozone_select)
```

```
## Joining with 'by = join_by(Date)'
```

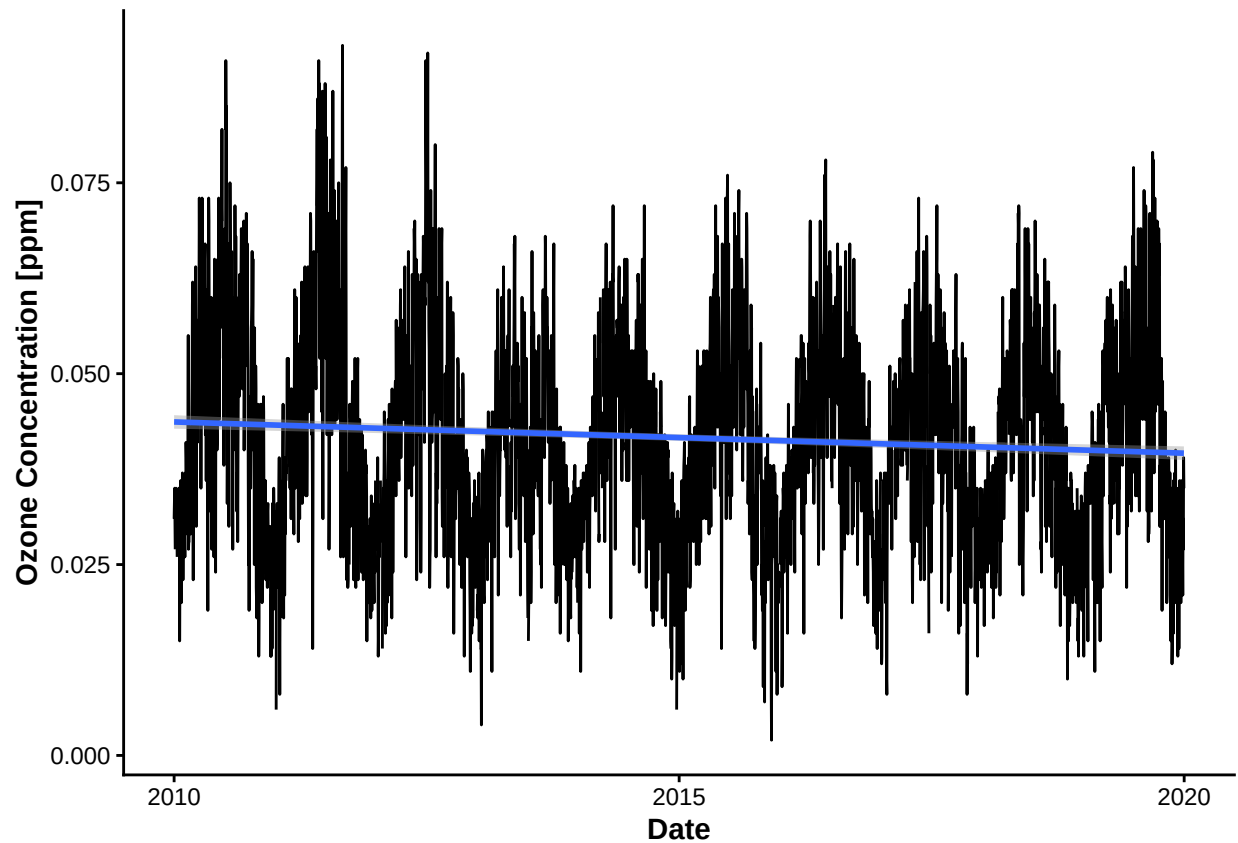
Visualize

7. Create a line plot depicting ozone concentrations over time. In this case, we will plot actual concentrations in ppm, not AQI values. Format your axes accordingly. Add a smoothed line showing any linear trend of your data. Does your plot suggest a trend in ozone concentration over time?

```
#7: line plot of ozone over time
ozone_plot <- ggplot(data = garinger_ozone,
  aes(x = Date, y = Daily.Max.8.hour.Ozone.Concentration)) +
  geom_line() +
  geom_smooth(method = 'lm') +
  labs(x = 'Date',
    y = "Ozone Concentration [ppm]")
ozone_plot
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: Removed 63 rows containing non-finite outside the scale range
## ('stat_smooth()').
```



Answer: Yes, there appears to be a decreasing linear trend and seasonality.

Time Series Analysis

Study question: Have ozone concentrations changed over the 2010s at this station?

8. Use a linear interpolation to fill in missing daily data for ozone concentration. Why didn't we use a piecewise constant or spline interpolation?

```
#8
summary(garinger_ozone) # Summary shows that there are 63 missing points

##      Date      Daily.Max.8.hour.Ozone.Concentration  DAILY_AQI_VALUE
##  Min.   :2010-01-01  Min.   :0.00200                Min.    : 2.00
## 1st Qu.:2012-07-01  1st Qu.:0.03200                1st Qu. : 30.00
## Median :2014-12-31  Median :0.04100                Median  : 38.00
## Mean   :2014-12-31  Mean   :0.04163                Mean    : 41.57
## 3rd Qu.:2017-07-01  3rd Qu.:0.05100                3rd Qu. : 47.00
## Max.   :2019-12-31  Max.   :0.09300                Max.    :169.00
##      NA's      :63
##      NA's      :63

# In real applications you will simply replace NAs
garinger_ozone <-
```

```
garinger_ozone %>%
  mutate( Daily.Max.8.hour.Ozone.Concentration = zoo::na.approx(Daily.Max.8.hour.Ozone.Concentration))
  mutate( DAILY_AQI_VALUE = zoo::na.approx(DAILY_AQI_VALUE))
summary(garinger_ozone) # Summary shows there are no missing points
```

```
##      Date      Daily.Max.8.hour.Ozone.Concentration DAILY_AQI_VALUE
## Min.   :2010-01-01   Min.   :0.00200               Min.    : 2.00
## 1st Qu.:2012-07-01   1st Qu.:0.03200               1st Qu. : 30.00
## Median :2014-12-31   Median :0.04100               Median  : 38.00
## Mean   :2014-12-31   Mean    :0.04151               Mean    : 41.41
## 3rd Qu.:2017-07-01   3rd Qu.:0.05100               3rd Qu. : 47.00
## Max.   :2019-12-31   Max.    :0.09300               Max.    :169.00
```

Answer: Piecewise constant assumes that the data are equal to the measurement made nearest to the date, and because there is an increasing and decreasing trend here, applying this interpolation may miss an inflection or change in the trend. A spline implements a quadratic function, so it would not be appropriate for the data here that exhibits a more linear trend.

9. Create a new data frame called `GaringerOzone_monthly` that contains aggregated data: mean ozone concentrations for each month. In your pipe, you will need to first add columns for year and month to form the groupings. In a separate line of code, create a new Date column with each month-year combination being set as the first day of the month (this is for graphing purposes only)

#9: Aggregated data

```
# Generate mean ozone concentration per month for each year
garinger_ozone_monthly <- garinger_ozone %>%
  mutate(Month = month(Date)) %>%
  mutate(Year = year(Date)) %>%
  group_by(Year, Month) %>%
  summarise(meanOzone = mean(Daily.Max.8.hour.Ozone.Concentration))
```

```
## 'summarise()' has grouped output by 'Year'. You can override using the
## '.groups' argument.
```

Add a date column and rearrange the order of the columns

```
garinger_ozone_monthly <- garinger_ozone_monthly %>%
  mutate( Date = my(paste0(Month,"-",Year))) %>%
  select(Date, Month, Year, meanOzone)
```

10. Generate two time series objects. Name the first `GaringerOzone.daily.ts` and base it on the dataframe of daily observations. Name the second `GaringerOzone.monthly.ts` and base it on the monthly average ozone values. Be sure that each specifies the correct start and end dates and the frequency of the time series.

#10: Generating time series objects

```
# Time series object for daily ozone observations from 2010-01-01 to 2019-12-31
garinger_ozone_daily_ts <- ts(data = garinger_ozone$Daily.Max.8.hour.Ozone.Concentration,
  start = c(year(first(garinger_ozone$Date)),
    month(first(garinger_ozone$Date))),
```

```

        day(first(garinger_ozone$Date)),
        end = c(year(last(garinger_ozone$Date)),
                month(last(garinger_ozone$Date)),
                day(last(garinger_ozone$Date))),
        frequency = 365)

# Time series object for monthly mean ozone observations from 2010-01 to 2019-12
garinger_ozone_monthly_ts <- ts(data = garinger_ozone_monthly$meanOzone,
    start = c(year(first(garinger_ozone_monthly$Date)),
              month(first(garinger_ozone_monthly$Date))),
    end = c(year(last(garinger_ozone_monthly$Date)),
            month(last(garinger_ozone_monthly$Date))),
    frequency = 12)

```

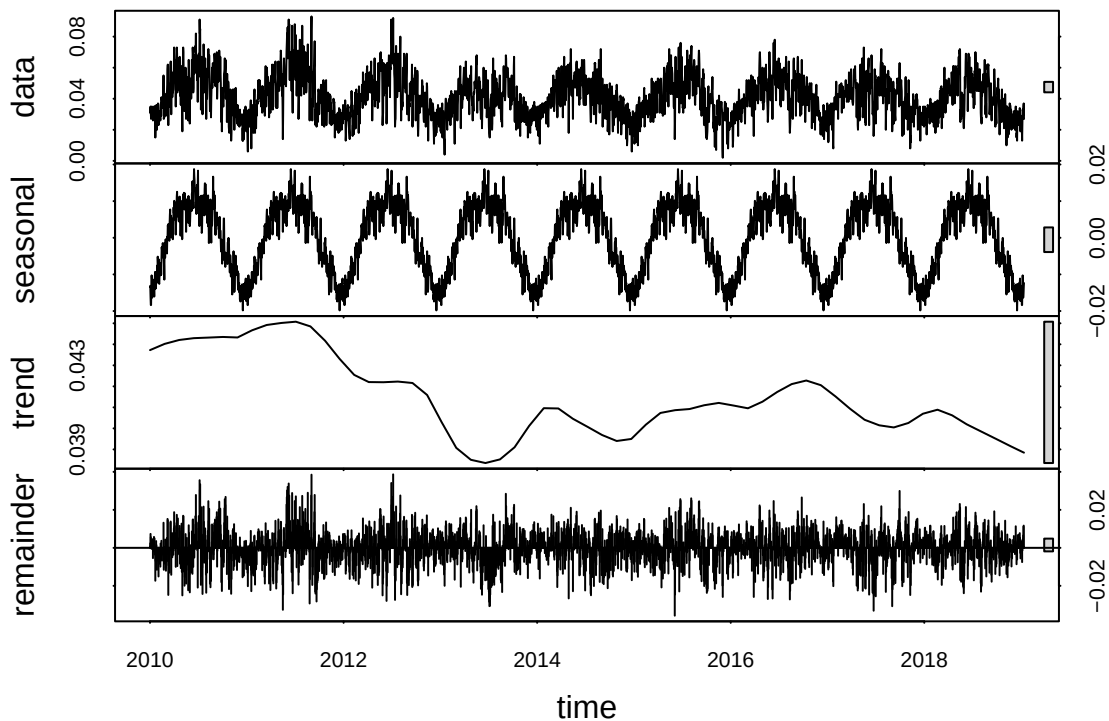
11. Decompose the daily and the monthly time series objects and plot the components using the `plot()` function.

```

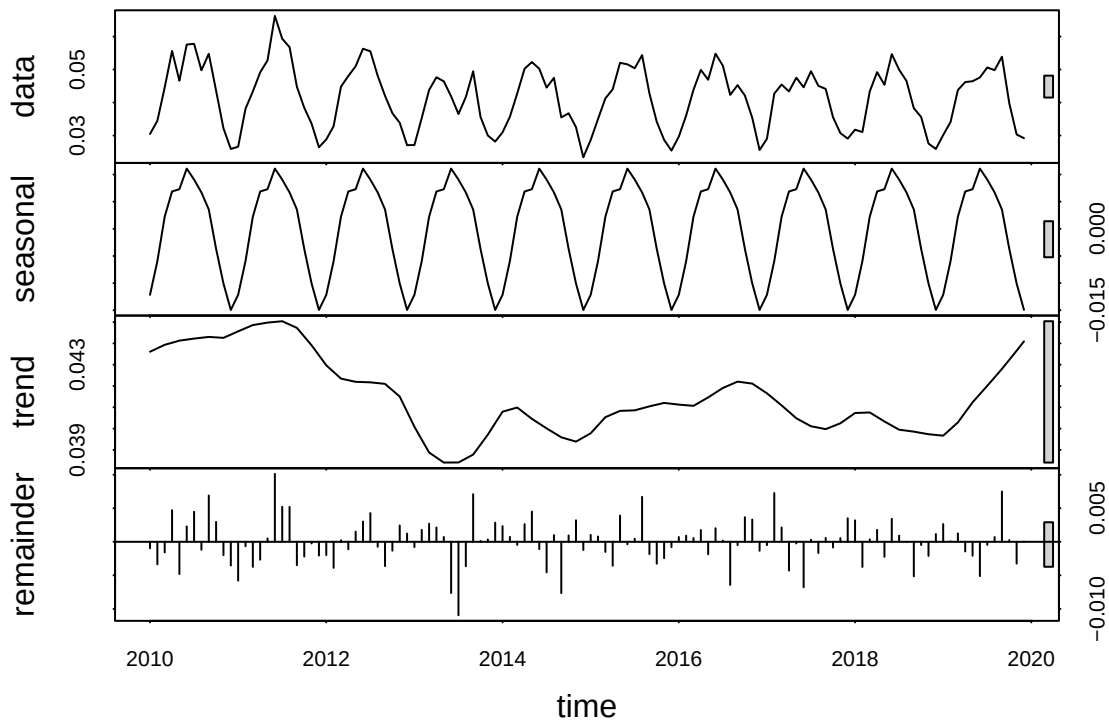
#11: Decomposing the time series

# Daily observations
garinger_ozone_daily_decomp <- stl(garinger_ozone_daily_ts, s.window = 'periodic')
plot(garinger_ozone_daily_decomp)

```



```
# Monthly observations
garinger_ozone_monthly_decomp <- stl(garinger_ozone_monthly_ts, s.window = 'periodic')
plot(garinger_ozone_monthly_decomp)
```



12. Run a monotonic trend analysis for the monthly Ozone series. In this case the seasonal Mann-Kendall is most appropriate; why is this?

```
#12: SMK test
```

```
# Monthly observations
```

```
garinger_ozone_monthly_smk <- trend::smk.test(garinger_ozone_monthly_ts)
garinger_ozone_monthly_smk
```

```
##
## Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
## data: garinger_ozone_monthly_ts
## z = -1.963, p-value = 0.04965
## alternative hypothesis: true S is not equal to 0
## sample estimates:
##      S varS
## -77 1499
```

```
summary(garinger_ozone_monthly_smk)
```

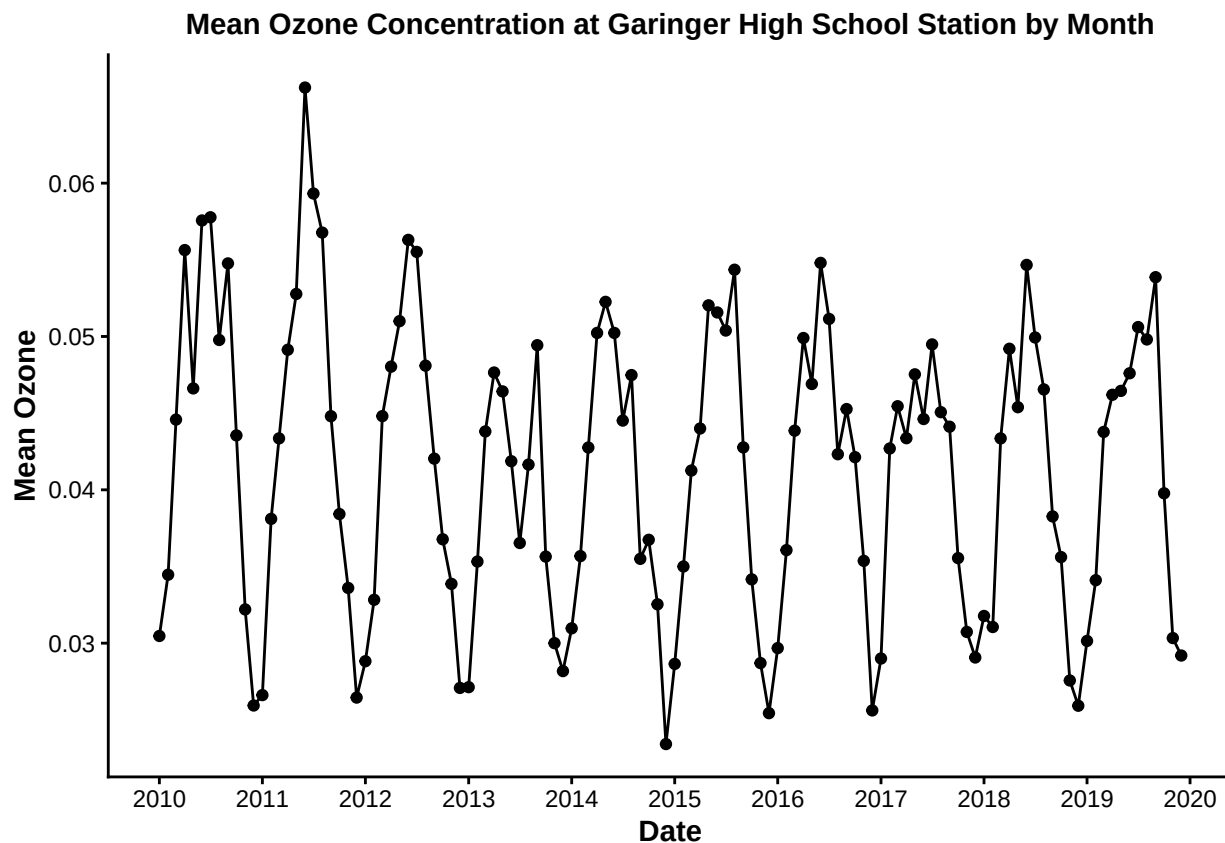
```
##
## Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
## data: garinger_ozone_monthly_ts
## alternative hypothesis: two.sided
##
## Statistics for individual seasons
##
## H0
##
##      S varS   tau      z Pr(>|z|)
## Season 1:  S = 0   15  125  0.333  1.252  0.21050
## Season 2:  S = 0   -1  125 -0.022  0.000  1.00000
## Season 3:  S = 0   -4  124 -0.090 -0.269  0.78762
## Season 4:  S = 0  -17  125 -0.378 -1.431  0.15241
## Season 5:  S = 0 -15  125 -0.333 -1.252  0.21050
## Season 6:  S = 0 -17  125 -0.378 -1.431  0.15241
## Season 7:  S = 0 -11  125 -0.244 -0.894  0.37109
## Season 8:  S = 0   -7  125 -0.156 -0.537  0.59151
## Season 9:  S = 0   -5  125 -0.111 -0.358  0.72051
## Season 10: S = 0 -13  125 -0.289 -1.073  0.28313
## Season 11: S = 0 -13  125 -0.289 -1.073  0.28313
## Season 12: S = 0  11  125  0.244  0.894  0.37109
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Answer: Based on the decomposition plot analysis, this is a clear seasonal trend in this data. Thus, the seasonal Mann-Kendall test is the only appropriate test because it is effective and capable of handling seasonal data.

13. Create a plot depicting mean monthly ozone concentrations over time, with both a `geom_point` and a `geom_line` layer. Edit your axis labels accordingly.

```
#13: Mean monthly ozone plot
```

```
mean_ozone_monthly_plot <- ggplot(garinger_ozone_monthly,
                                   aes(x = Date,
                                       y = meanOzone)) +
  geom_line() +
  geom_point() +
  labs(x = "Date",
       y = "Mean Ozone",
       title = "Mean Ozone Concentration at Garinger High School Station by Month") +
  scale_x_date(date_labels = "%Y",
               date_breaks = "1 year")
mean_ozone_monthly_plot
```

14. To accompany your graph, summarize your results in context of the research question. Include output from the statistical test in parentheses at the end of your sentence. Feel free to use multiple sentences in your interpretation.

Answer: Based on the decomposed graph, at this station, ozone concentrations appear to have seasonal fluctuations that peak in the middle of the year (graph). However, the statistical analysis suggest that there is not a strong monotonic trend observed in the data (p-value = 0.04965).

15. Subtract the seasonal component from the `GaringerOzone.monthly.ts`. Hint: Look at how we extracted the series components for the `EnoDischarge` on the lesson Rmd file.
16. Run the Mann Kendall test on the non-seasonal Ozone monthly series. Compare the results with the ones obtained with the Seasonal Mann Kendall on the complete series.

#15

Create a dataframe of the components

```
garinger_ozone_components <- as.data.frame(garinger_ozone_monthly_decomp$time.series[,1:3])
```

Subtract the seasonal component from the timeseries

```
garinger_ozone_noseason <- garinger_ozone_monthly_ts - garinger_ozone_components$seasonal
```

#16

```
garinger_ozone_noseason_mk <- trend::mk.test(garinger_ozone_noseason)
garinger_ozone_noseason_mk
```

```
##
## Mann-Kendall trend test
##
## data: garinger_ozone_noseason
## z = -2.672, n = 120, p-value = 0.00754
## alternative hypothesis: true S is not equal to 0
## sample estimates:
##          S          varS          tau
## -1.179000e+03  1.943657e+05 -1.651376e-01
```

```
summary(garinger_ozone_noseason_mk)
```

```
##          Length Class  Mode
## data.name    1      -none- character
## p.value      1      -none- numeric
## statistic    1      -none- numeric
## null.value   1      -none- numeric
## parameter    1      -none- numeric
## estimates    3      -none- numeric
## alternative  1      -none- character
## method       1      -none- character
## pvalg        1      -none- numeric
```

Answer: The Mann-Kendall test when seasonality is removed still suggests that there is a significant monotonic trend present in the dataset (p-value = 0.00754). Overall, this study suggests that when seasonality is removed, there is a trend in the data but this is not observed when seasonality is included.